



Universitat
Oberta
de Catalunya

(DBPHP) Desarrollo back-end con PHP
PEC 3 - Aprendamos a programar en PHP
(Explicaciones)

Marc Turu Roca

Profesor: Samuel González Rodríguez

Fecha de entrega: 17/12/2025

Universitat Oberta de Catalunya (UOC)

Tabla de contenidos

0. Introducción	2
A1. Creación de la base de datos	2
A2. Acceso a la base de datos	4
A3. Creación de la página de inicio	5
A4. Creación de las páginas para los museos	6
A5. Ordenación y filtro de contenidos	8
A6. Menú	9
A7. API	10
A8. Login	11
A9. Alta y edición de usuarios	12
A10. Publicación	13
11. Estilos y adiciones	14
11.1 Base de datos	14
11.2 Museo aleatorio y post	14
11.3 Home	14
11.4 Museos	14
11.5 Menú	15
11.6 APIs	15
11.7 Login, signup y edit	15
11.8 Adición de museos	15

0. Introducción

La PEC 3 consistía en construir un sitio web que sirviera como gestor de museos/exposiciones almacenados en una BD real **MySQL** en PhpMyAdmin de WAMP, programado con **PHP** puro y apoyándose en **HTML** y **CSS**.

Este proyecto se centró en la creación de un Directorio de Museos y Exposiciones con diferentes vistas, opciones de filtros, registro e inicio de sesión,... por lo que se usaron y trataron una gran variedad de elementos del lenguaje en cuestión.

A continuación se sintetizan las **explicaciones** de las implementaciones de las **actividades** en el entorno local para después probar su funcionamiento en el servidor. Estas explicaciones se presentan siguiendo el flujo lineal del desarrollo de cada actividad, reflejando de manera concreta la **resolución** de las mismas. Las adiciones posteriores se comentan en **11. Estilos y adiciones**, por lo que las explicaciones muestran la experiencia y el desarrollo tal como se abordó en cada etapa, incluyendo errores y pasos realizados.

A1. Creación de la base de datos

Para el inicio de esta PEC, a diferencia de la anterior donde se tuvo que utilizar la herramienta gestora de contenidos *Drupal* con *sqlite*, se optó por el uso de *MySQL*, por lo que se creó una BD en *PhpMyAdmin*, con nombre **dbphppec3_db**.

Para entender mejor cómo hacerlo, se utilizó el siguiente recurso de *YouTube*: https://www.youtube.com/watch?v=0zz89bSh0b0&list=PLH_tVOsiVGzmnI7ImSmhlw5qb9Sy5KJRE&index=57.

En la creación de la BD, se definió el nombre, el cotejamiento y se creó la tabla **museums_museum** con los siguientes campos: id, nombre, ciudad, tematica, fechas_horarios, visitas_guiadas, precio e imagen.

- **id** se creó como INT 10 porque sería el entero autoincremental (AI) que identificaría al museo (PRIMARY KEY).
- **nombre** se creó como VARCHAR 40 para la longitud necesaria.
- **ciudad** se creó como VARCHAR 20 porque la longitud suele ser más corta.
- **tematica** se creó como ENUM porque dependería de un rango cerrado ya utilizado en la PEC2 (arte, familia, historia, tecnología, temporal, tradición) para los filtros.
- **fechas_horarios** se creó como TEXT, como se especificaba.
- **visitas_guiadas** se creó como ENUM porque se el valor debía ser "sí" o "no".
- **precio** se creó como DECIMAL 6,2.
- **imagen** se creó como VARCHAR 255, ya que se guardaría la ruta y el nombre.



Fig. 1 - ENUM de temática con valores posibles

Finalmente, se insertaron en la BD 10 museos, 3 de los cuales eran reales (Museo del Prado, Museu Picasso i Guggenheim Bilbao), con la siguiente consulta SQL:

```
INSERT INTO museums_museos
(nombre, ciudad, tematica, fechas_horarios, visitas_guiadas, precio, imagen)
VALUES
('Museo del Prado', 'Madrid', 'arte',
'Lun-Sab 10:00-20:00; Dom 10:00-19:00', 'si', 15.00, '/img/prado.jpg'),
('Museu Picasso', 'Barcelona', 'arte',
'Mar-Dom 10:00-19:00', 'si', 12.00, '/img/picasso.jpg'),
('Guggenheim Bilbao', 'Bilbao', 'arte',
'Mar-Dom 11:00-19:00', 'si', 16.00, '/img/guggenheim.jpg'),

('Museo Marítimo Vlc', 'Valencia', 'historia',
'Lun-Dom 09:00-18:00', 'no', 8.50, '/img/maritimo.jpg'),
('Museo Ciencias Med', 'Alicante', 'tecnologia',
'Mar-Dom 10:00-20:00', 'si', 10.00, '/img/naturales.jpg'),
('Centro Arte Nova', 'Barcelona', 'arte',
'Lun-Vie 10:00-18:00', 'si', 14.00, '/img/nova.jpg'),
('Museo Pirineo', 'Huesca', 'historia',
'Mar-Sab 10:00-17:00', 'no', 7.00, '/img/pirineo.jpg'),
('Museo Mediterrani', 'Tarragona', 'tradicion',
'Lun-Dom 09:30-19:00', 'si', 9.00, '/img/mediterranea.jpg'),
('Museo Espacio MX', 'Madrid', 'tecnologia',
'Lun-Sab 10:00-21:00', 'si', 11.50, '/img/espacio.jpg'),
('Museo Solaris Art', 'Valencia', 'arte',
'Lun-Dom 10:00-20:00', 'no', 13.00, '/img/solaris.jpg');
```

A2. Acceso a la base de datos

Para esta segunda actividad, antes de empezar a programar, se organizaron las jerarquías de archivos del proyecto, separando la `/api`, `/assets`, `/config`, `/includes` y los archivos `.php` pertinentes para, posteriormente, empezar a programar.

Seguidamente, se creó **`db_config.php`** para iniciar la conexión a la BD con el método **PDO**:

```
$conn = new PDO("mysql:host=$DB_HOST;dbname=$DB_NAME", $DB_USER, $DB_PASS);
```

Con los valores predefinidos:

```
$DB_HOST = "localhost";  
$DB_NAME = "dbphppec3_db";  
$DB_USER = "root";  
$DB_PASS = "";
```

De esta manera, un único archivo podía importarse y utilizarse en los demás para realizar consultas a la BD.

Para finalizar esta actividad, se creó un script **`random-museum.php`** en el cual se mostraba un museo aleatorio de los 10 disponibles en la BD. Para hacerlo, se combinó el uso de PHP y SQL:

```
$stmt = $conn->query("SELECT * FROM museums_museos ORDER BY RAND() LIMIT 1");  
1
```

con PHP incrustado en HTML para la muestra de los atributos del resultado.

Este HTML era un *unordered-list* con cada *list item* mostrando el atributo respectivo con

```
<?php echo htmlspecialchars($museum['nombre']); ?>  
2 ...
```

También se usa un `<head>` en HTML para añadir la siguiente información para cada página:

```
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <meta name="description" content="Página que muestra un museo de manera  
aleatoria.">  
    <title>Directorio de Museos y Exposiciones filtrable</title>  
</head>
```

¹ Estas consultas SQL se encapsulan en `try { $stmt ... } catch (PDOException $e) { die ... }`

² Se usa `htmlspecialchars` para convertir caracteres especiales en entidades HTML, evitando que el navegador interprete contenido dinámico como código ejecutable. De esta manera, se asegura una visualización segura de los datos obtenidos de la BD.

A3. Creación de la página de inicio

Para esta actividad, se creó un archivo *index.php* (que serviría como referencia base de la web, es decir, de página inicial por defecto) y se organizó su estructura de la siguiente manera:

Inicialmente, para la interacción con la BD, se utilizaron consultas SQL encapsuladas dentro de PHP para obtener los dos museos reales fijos y los tres museos ficticios, los cuales deberían rotar aleatoriamente en cada recarga de la página:

```
$stmt = $conn->query("SELECT id, nombre, ciudad, precio, imagen FROM museums_museos WHERE id IN (1,2)");  
$stmt = $conn->query("SELECT id, nombre, ciudad, precio, imagen FROM museums_museos WHERE id NOT IN (1,2) ORDER BY RAND() LIMIT 3");
```

Para esta segunda consulta se considera relevante comentar el `NOT IN (1,2)` para excluir los museos reales ya incluidos en la primera consulta.

A continuación, y siguiendo la estructura estándar, se mostró el listado de los cinco museos por pantalla usando PHP dentro de HTML. Para mostrar los atributos se hizo de la misma manera que para la **A2. Acceso a la base de datos**, pero esta vez usando bucles:

```
<?php foreach($real_museums as $real_museum): ?>
```

```
<?php foreach($fake_museums as $fake_museum): ?>
```

porque la consulta SQL contenía un array de museos a mostrar.

Cada museo incluía un link en el mismo nombre para redirigir al usuario a la página individualizada de dicho museo, y la incorporación de una imagen por *default* en caso de no encontrar la imagen propia del museo en la BD:

```
">  
  
<p><a href="post.php?id=<?php echo $real_museum['id']; ?>"> <?php echo  
htmlspecialchars($real_museum['nombre']); ?></a></p>
```

A4. Creación de las páginas para los museos

Para esta actividad un poco más extensa, inicialmente se creó un archivo ***museums.php***, en el cual se llevaron a cabo los siguientes pasos:

- Declaración de variables para la paginación:

- Conseguir por parámetro el número de *page* actual.

```
$page = isset($_GET['page']) ? (int)$_GET['page'] : 1;  
$page = max(1, $page);
```

- Establecer el límite de museos por página.

```
$limit = 5;
```

- Calcular el *offset* para la *query* de la consulta SQL.

```
$offset = ($page - 1) * $limit;
```

- Añadir la nueva información como declaración al SQL base predefinido:

```
$sqlBase .= " LIMIT :limit OFFSET :offset";
```

- *Fetch* de los museos para la página actual:

- Obtener el número total de museos.

```
$stmt = $conn->query("SELECT count(*) FROM museums_museos");  
$numOfMuseums = $stmt->fetchColumn();
```

- Obtener los museos de la página actual.

```
$stmt = $conn->prepare("SELECT id, nombre, ciudad, precio, imagen  
FROM museums_museos ORDER BY id ASC LIMIT :limit OFFSET  
:offset");  
$stmt->bindValue(':limit', $limit, PDO::PARAM_INT);  
$stmt->bindValue(':offset', $offset, PDO::PARAM_INT);  
$stmt->execute();  
$museums = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

En este contexto, se usaron `prepare` y `bindValue` antes de ejecutar la consulta para que los valores se envíen separadamente de la consulta para prevenir que alguien externo cambie los valores para modificar la consulta SQL. También se usó porque, como calculamos los `$limit` y `$offset` que elige el usuario en el *script*, se asignan marcadores con `:` para asignarles los valores después, al ejecutar la consulta

- Finalmente, se mostraron los mismos campos que los de la página de inicio con la adición de la paginación en un contenedor separado:

```
<div>
    <?php for ($i = 1; $i <= $totalPages; $i++): ?>
        <a href="?page=<?php echo $i; ?>"><?php echo $i; ?></a>
    <?php endfor; ?>
</div>
```

En segundo lugar, se creó un archivo **post.php**, en el cual se llevaron a cabo los siguientes pasos:

- Declaración del *id* relacionado con el museo de esa página:
 - Conseguir por parámetro el número de *id* actual (si existe y no es nulo).

```
if (!isset($_GET['id'])) die("ID del museo no especificado");
```

- Asignar el *id* obtenido a la variable *\$id*.

```
$id = (int)($_GET['id']);
```

- *Fetch* del museo para la *id* actual:

- Obtener el museo del *\$id*.

```
$stmt = $conn->prepare("SELECT * FROM museums_museos WHERE id = :id");
$stmt->bindValue(':id', $id, PDO::PARAM_INT);
$stmt->execute();
$museum = $stmt->fetch(PDO::FETCH_ASSOC);
```

Se usó el mismo contexto que antes para los *prepare* y *bindValue*.

- Finalmente, se mostraron todos los campos del museo en cuestión de una manera similar a como se hizo con *random-museum.php*.

Además de realizar estas funcionalidades, los *links* que enlazaban cada museo con su página individual (*post*) también se añadieron en el nombre de cada uno en la página de museos (como se hizo con la de inicio):

```
Nombre:
<a href="post.php?id=<?php echo $museum['id']; ?>">
    <?php echo htmlspecialchars($museum['nombre']); ?>
</a>
```

A5. Ordenación y filtro de contenidos

Para esta actividad, se tuvieron que realizar las siguientes modificaciones respecto al primer *museums.php*:

- Gestión de filtros:
 - Definir las variables de filtrado (orden y temática).

```
$order = 'ASC';  
if (isset($_GET['order']) && strtolower($_GET['order']) ===  
'desc') $order = 'DESC';  
$hasOrder = isset($_GET['order']);  
  
$theme = isset($_GET['theme']) ? $_GET['theme'] : null;
```

- Determinar la necesidad de filtrado con `$needPagination`.

```
$needPagination = !$hasOrder && !$theme;
```

- Construcción dinámica de la consulta SQL:

- Declarar la consulta SQL base (para cualquier escenario).

```
$sqlBase = "SELECT id, nombre, ciudad, precio, imagen FROM  
museums_museos";
```

- Añadir las cláusulas extras (si así se requería) a la consulta SQL base.

```
if ($theme) {  
    $sqlBase .= " WHERE tematica = :theme";  
}  
if ($hasOrder) {  
    $sqlBase .= " ORDER BY precio $order";  
} else {  
    $sqlBase .= " ORDER BY id ASC";  
}
```

A parte de estos cambios, el código también incorporó la paginación (por si fuera necesaria) cuando se accediera a la página sin filtros, donde se añadía el *limit* y *offset* a la consulta. Esta se hizo con el un bloque de código accesible con `if ($needPagination)`.

Además, se preparó la consulta, se hicieron los `bindValue` solamente si las variables previamente definidas no eran nulas³, se ejecutó la consulta final y, finalmente, se añadieron los `?order=asc">Ascendente` , `?theme=Arte">Arte` ,... para cada filtro y el `<?php if ($needPagination): ?>` para controlar la muestra de paginación por pantalla en el HTML.

³ No se hizo el `bindValue` en `$order` porque este se inserta directamente al ser validado al inicio.

A6. Menú

Para esta actividad, se decidió utilizar código reutilizable y se crearon dos archivos nuevos: **header.php** y **menu.php**. El primero, al igual que con *db_config.php*, se importaría en cada página que fuera necesario.

El flujo de trabajo fue:

- Creación de *menu.php*:
 - Añadir un menú de navegación con la etiqueta `<nav>` y crear una *unordered-list* de cada elemento mencionado en el **R3** que debía estar presente en el menú.
 - Controlar la sesión activa del usuario (para saber qué botones mostrar):

```
<?php if (!isset($_SESSION['username'])): ?>
```

- Creación de *header.php*:
 - Añadir el menú de navegación creado al *header.php*:

```
<?php require_once __DIR__ . '/menu.php'; ?>
```

Se utilizó `require_once` porque carga el archivo una única vez y detiene la ejecución si el archivo falla.

Se utilizó `__DIR__` porque devuelve la ruta absoluta del archivo actual, y como *header.php* está en */includes/* (*menu.php*), es lo que se necesita.

Se utiliza `.` para concatenar *strings*.

- Adición del *header* completo:
 - Hacer un

```
require_once "includes/header.php";
```

para añadir el archivo donde se ha construido el *header* común.

De esta manera, se podría reutilizar el código de la carpeta */includes/* para partes de código que muchos archivos comparten como, por ejemplo, este caso del *menú* y *header*.

A7. API

Para esta actividad, se crearon dos puntos de entrada de solo lectura en el directorio `/api`:

1. `/api/museums.php (/api/museums/<page>)`. Devuelve un listado de museos en formato *JSON*. La paginación se controlaba mediante el parámetro `page` pasado por URL, configurando un máximo de 10 museos por página.
2. `/api/museum.php (/api/museum/<id>)`. Devuelve toda la información de un museo en formato *JSON*. El museo se seleccionaba mediante el parámetro `id` pasado por URL.

Debido a las condiciones de acceso, se configuró un `.htaccess inicial` en el directorio `/api`, redirigiendo las peticiones a los scripts PHP correspondientes:

```
RewriteEngine on
# api/museums/1 -> museums.php?page=1
RewriteRule ^museums/([0-9]+)/?$ museums.php?page=$1 [L,QSA]
# api/museum/1 -> museum.php?id=1
RewriteRule ^museum/([0-9]+)/?$ museum.php?id=$1 [L,QSA]
```

Así pues, Apache permitía que los parámetros dinámicos fueran accesibles mediante `$_GET`.

En los *scripts* de la API, se validó la existencia de los parámetros `page` e `id` mediante `isset()`, se construyeron las consultas SQL y los resultados obtenidos se devolvieron en formato JSON utilizando `echo json_encode($museums);`.

Para que las APIs se abrieran en una nueva pestaña del navegador y permitir la selección dinámica de página o museo, se modificó el menú con `label`, `input` y `button`, usando `window.open` para generar la URL según el valor introducido. De este modo, se conseguía libertad de elección, cumplir con el requisito y evitar el uso de parámetros GET como `?id=1`.⁴

Durante esta actividad surgió un primer problema: las rutas se escribieron con una `/` inicial que provocaba que el navegador buscara desde la raíz, ignorando la base del proyecto. Se solucionó utilizando rutas relativas, eliminando la barra inicial de `/api/museum/1`.

Otro problema fue que las reglas de reescritura no se aplicaban correctamente, provocando que los parámetros dinámicos de la URL no llegaran a los *scripts*. Tras confirmar que `AllowOverride all` estaba activo en `httpd.conf`, se identificó que `MultiViews`, activada por defecto en la configuración Apache, interceptaba las peticiones antes de que fueran procesadas por el `rewrite`. Se solucionó desactivando la opción `MultiViews` en el contexto de la API (en `.htaccess`).

⁴ Con HTML puro, solo es posible crear enlaces fijos (`<a href="..." target="_blank">`), pero no se puede modificar la URL según el valor que el usuario introduzca en el `input`.

A8. Login

Esta actividad se organizó en tres bloques diferenciados dentro de *login.php*:

1. Creación de la tabla `museums_user`.

Se definió una nueva tabla en el servidor (*back-end*) con los campos: *username* (*PRIMARY-KEY*), *name*, *surname* y *password*.

2. Prevención contra *SQL injection*.

Para evitar ataques peligrosos que vulneren la seguridad de los datos de los usuarios si se ejecutan consultas SQL hacia la BD con *inputs* maliciosos, se llevaron a cabo métodos de prevención como:

2.1 Validación de datos del input.

2.2 Preparación de la consulta (`username = :username`).

2.3 Preparación de la declaración (`$conn->prepare(...)`).

2.4 Enlace de los parámetros (`$stmt->bindValue(...)`).

2.5 Ejecución de la consulta (`$stmt->execute();`).

2.6 Obtención de los resultados (`$stmt->fetch(PDO::FETCH_ASSOC)`).

Y otro con la descryptación del *password* de la BD con `password_verify($password, $user['password'])`.⁵

3. Construcción del formulario.

Con la nueva tabla creada y las prevenciones para evitar *SQL injection* y otros ataques, se definió la estructura del formulario para que el usuario se identificara (después de crear la lógica de recogida de estos datos en PHP (`$_POST`) y la lógica de inserción del usuario en la tabla `museums_users` con la consulta SQL).

Esta era un `<form method="post">` con dos pequeños bloques de `label` y `input` con `username (required)` y `password (required)`, y un `<button type="submit">Iniciar sesión</button>` final.

Además, se manejaron los mensajes de error mediante una variable global que los iba recogiendo en las comprobaciones de la lógica PHP y se mostraba en el HTML:

```
<?php if ($message): ?>
    <p class="message"><?php echo htmlspecialchars($message); ?></p>
<?php endif; ?>
```

En otro archivo *logout.php*, se cerraba la sesión y redirigía al usuario a la página de inicio.

⁵ La encriptación del *password* se comenta en el siguiente punto **A9. Alta y edición de usuarios**.

A9. Alta y edición de usuarios

Esta actividad iba muy relacionada con la **A8.Login** ya que, en la nueva página **signup.php**, a parte de todas las prevenciones y comprobaciones comentadas anteriormente para asegurar el correcto funcionamiento de la funcionalidad, al igual que el requerimiento de insertar todos los campos, se incluyó la función `password_hash($password, PASSWORD_BCRYPT);`, que se encargaba de encriptar la contraseña que el usuario introduciría en el *form*.

Este *form* iba incluido de una manera similar que en la actividad anterior, añadiendo esta vez un `placeholder="Usuario",...` para cada campo.

En segundo lugar, para la edición del perfil de usuario, aparte de seguir la misma estructura para la lógica PHP (recogida de datos del *form*, validación de los *inputs*,...), se presentaron dos posibles escenarios:

- Actualización del perfil de usuario *incluyendo* la contraseña.
Para este primer caso, se usaba de nuevo la `password_hash($password, PASSWORD_BCRYPT);` para proteger la contraseña y se actualizaban los campos del usuario obtenido con la consulta SQL.
- Actualización del perfil de usuario *sin incluir* la contraseña.
Para este segundo caso, se actualizaban los campos exceptuando la contraseña.

Estas ejecuciones (que iban siempre sucedidas por el *bindeo* de los valores de la consulta y su ejecución), se respaldaban con las siguientes consultas SQL:

1. `UPDATE museums_users SET value = :value WHERE username = :username` para la actualización (**edición**).
2. `INSERT INTO museums_users (username, name, surname, password) VALUES (:username, :name, :surname, :password)` para la inserción (**alta**).
3. `SELECT * FROM museums_users WHERE username = :username` para la selección (**login**).

Finalmente, se registró un usuario inicial con⁶:

- **username:** mturur
- **name:** Marc
- **surname:** Turu
- **password:** mturur

⁶ Este registro se podía hacer desde la propia BD en *PhpMyAdmin* o el botón de *Sign up* del menú.

A10. Publicación

1. Primeramente, me dirigí a http://localhost/phpmyadmin/index.php?route=/database/export&db=dbphppec3_db i exporté la BD como SQL y, después de comprimirla, le cambié el nombre para que terminara en **.sql.zip**.
2. En segundo lugar, comprimir el proyecto de la PEC3 que tenía en mi equipo (**C:\wamp64\www\dbphppec3_museums**).
3. Con las dos compresiones hechas, accedí al PhpMyAdmin del servidor remoto (utilizando las mismas credenciales que para acceder al servidor de pruebas) a través de la siguiente URL: <https://eimtcms2.techlab.uoc.edu/bbdd/>.
4. Ya dentro, importé la BD que exporté en el punto 1 (dentro de la BD ya creada **mturur**).
5. Después de crear la carpeta **pec3/** en el servidor remoto, copié el .zip del proyecto en esa ubicación.
6. Accedí al servidor por conexión **ssh** en la terminal hasta mi carpeta de **/pec3** y descomprimí el .zip en cuestión⁷.
7. Seguidamente, edité cada archivo pertinente desde **FileZilla** para arreglar:
 - a. Los problemas de credenciales de acceso a la nueva BD de <https://eimtcms2.techlab.uoc.edu/bbdd/>.

```
$DB_NAME = "mturur";  
$DB_USER = "mturur";  
$DB_PASS = "mi_contraseña_al_servidor";
```
 - b. Las rutas relativas de las imágenes (para añadir el prefijo delante de la ruta de la imagen en la BD):

```
$img_prefix = "/~mturur/pec3";
```
 - c. y la ruta del **.htaccess**:

```
RewriteBase /~mturur/pec3/dbphppec3_museums/api/
```
9. Una vez estos problemas estaban resueltos, probé que todas las funcionalidades de la PEC 3 funcionasen correctamente, como lo hacían en local.

⁷ Este se eliminó posteriormente por motivos de seguridad.

11. Estilos y adiciones

Una vez realizadas las actividades propuestas y tratados los requisitos de cada una, se hicieron algunas variaciones de código para remodelar el aspecto visual (mayoritariamente) del sitio web para que tuviera mejor organización y fuera más agradable visualmente.

11.1 Base de datos

Para la BD, tuve que editar el campo **imagen** de cada museo para añadir delante de cada ruta: `/dbphppec3_museums/assets`.

El campo **visitas_guiadas** podría haber sido un *boolean* para después relacionarlo con "Sí" o "No", pero se optó por dejar ENUM.

11.2 Museo aleatorio y post

Se decidió pasar de la utilización de una *unordered-list* a dos `<div class="museum-image">` y `<div class="museum-details">` que separaban la página en dos secciones.

11.3 Home

Los atributos de cada `if ($real_museums):` y `if ($fake_museums):` se mostraron dentro en dos `<div class="museums-row">` separados (dependiendo de la clase de museo) y los detalles, utilizando `<p>` dentro de cada `<div class="museum-card">`.

11.4 Museos

Se mejoró la manera en que se obtenía el orden para el precio:

```
$order = null;
$hasOrder = false;
if (isset($_GET['order']) && in_array(strtolower($_GET['order']),
['asc', 'desc'])) { ... }
```

Se añadió la siguiente comprobación, siendo `allowedThemes` las temáticas de la web:

```
$theme = (isset($_GET['theme']) && in_array($_GET['theme'],
$allowedThemes)) ? $_GET['theme'] : null;
```

11.5 Menú

Para marcar de manera visual en qué página se encontraba el usuario, se utilizó:

```
$current_page = basename($_SERVER['PHP_SELF']); y class="<?php echo  
($current_page === 'index.php') ? 'active' : '' ; ?>"
```

11.6 APIs

Para este apartado se añadió el `RewriteBase /dbphppec3_museums/api/` en el `.htaccess` y se modificaron los `label`, `input`, `button` y `window.open` del menú con:

```
<span>API_museums</span>  
<input type="number" id="museumsPage" min="1" value="1">  
<button type="button" onclick="window.open('api/museums/' +  
document.getElementById('museumsPage').value, '_blank')">Abrir</button>
```

El usuario podía elegir qué página o id de las APIs mostrar (con 1 por defecto).

Para `museum.php`, se manejaron los errores con códigos de respuesta HTTP.

- **400**: Parámetro ausente.
- **404**: Museo no encontrado.
- **500**: Error SQL.

11.7 Login, signup y edit

Para estas 3 vistas, se decidió usar una estructura similar en forma de *modal* a partir de un *div* que encapsulaba el *form* utilizado para la recopilación de datos.

Este *modal* estaba estructurado de la siguiente manera: *h1*, [{*label*, *input*}, ...] y *button*.

De este modo, se podía reciclar el código CSS para aplicar unos mismos estilos y mantener un diseño de formas y colores coherente en toda la aplicación.

11.8 Adición de museos

Se decidió añadir 1 museo extra (ficticio) para poder probar cómo la página se adaptaba al cambio de tener >10 museos en la BD:

1. **api/museums/2**. Mostraba correctamente los datos *JSON* de *únicamente* el museo extra (ya que se configuró 10 museos por página).
2. **museums.php?page=3**. Mostraba correctamente la tarjeta de *únicamente* el museo extra (ya que se configuraron 5 museos por página).